El motor de probabilidades (RNG) en un sistema tipo gacha o de apertura de sobres no se basa en una aleatoriedad pura, sino en un sistema de Pesos (Weighted RNG). Si un sobre tiene 5 cartas, el sistema no elige 5 autos al azar de toda la base de datos, porque de lo contrario los vehículos legendarios saldrían con demasiada frecuencia.

La estrategia profesional para presentar al jurado se divide en dos pasos: 1. Sortear la Rareza y 2. Sortear el Vehículo dentro de esa rareza.

Modelos en SQLAlchemy

Primero, definimos cómo se estructuran las tablas en models.py

```
from sqlalchemy import Column, Integer, String, Boolean, ForeignKey
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class VehicleCatalog(Base):
    __tablename__ = 'vehicles_catalog'
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    category = Column(String)
    rarity_tier = Column(String)  # 'C', 'B', 'A', 'S'

class UserInventory(Base):
    __tablename__ = 'user_inventory'
    user_id = Column(String, primary_key=True)
    vehicle_id = Column(Integer, ForeignKey('vehicles_catalog.id'), primary_key=True)
    is_pasted = Column(Boolean, default=False)
    unpasted_count = Column(Integer, default=0)
```

2. El Algoritmo de Drop Rate (rng_engine.py)

Para este MVP, configuramos los pesos de la siguiente manera:

Tier C (Urbano/Común): 70% de probabilidad.

Tier B (Deportivo/Raro): 20% de probabilidad.

Tier A (Superdeportivo/Épico): 9% de probabilidad.

Tier S (Hipercoche/Legendario): 1% de probabilidad.

Utilizamos la función random.choices nativa de Python, que está optimizada exactamente para cálculos de probabilidad con pesos.

```
import random
from sqlalchemy.orm import Session
from models import VehicleCatalog, UserInventory

# Configuración de los Drop Rates
TIER_WEIGHTS = {
    "C": 70.0,
    "B": 20.0,
    "A": 9.0,
    "S": 1.0
}

def generate_pack_drop(db: Session, pack_size: int = 5) -> list:
    """
    Algoritmo central para la apertura de una Caja de Importación.
    Retorna una lista con los IDs de los 5 vehículos obtenidos.
    """
    tiers = list(TIER_WEIGHTS.keys())
    weights = list(TIER_WEIGHTS.values())

    # 1. Sortear las 5 rarezas basándose en los pesos
    # Resultado de ejemplo: ['C', 'C', 'B', 'C', 'A']
    rolled_tiers = random.choices(tiers, weights=weights, k=pack_size)

    obtained_vehicles = []
```

```
    # 2. Por cada rareza sorteada, buscar un auto aleatorio de esa categoría
    for tier in rolled_tiers:
        # Obtenemos todos los autos de este tier en particular
        vehicles_in_tier = db.query(VehicleCatalog).filter(VehicleCatalog.rarity_tier == tier).all()

        if vehicles_in_tier:
            # Seleccionamos uno al azar de la lista filtrada
            chosen_vehicle = random.choice(vehicles_in_tier)
            obtained_vehicles.append(chosen_vehicle)

    return obtained_vehicles
```

3. La Transacción ACID de Apertura (unboxing.py)

Al jurado le interesará ver cómo unes el algoritmo matemático con la integridad de los datos en SQLite. Cuando el usuario hace POST /api/un-crate, se ejecuta esta lógica:

```
from sqlalchemy.orm import Session
from fastapi import HTTPException
from models import UserInventory
# Asumiendo que generate_pack_drop está importado

def open_crate_transaction(db: Session, user_id: str):
    try:
        # 1. Validar que el usuario tenga cajas o créditos suficientes (omitido para brevedad)

        # 2. Generar el Drop (Los 5 vehículos)
```

```
new_vehicles = generate_pack_drop(db, pack_size=5)


# 3. Inyectar los vehículos en el inventario dentro de una transacción
for vehicle in new_vehicles:
    # Buscar si el usuario ya tiene esta figurita (duplicada)
    inventory_item = db.query(UserInventory).filter_by(
        user_id=user_id,
        vehicle_id=vehicle.id
    ).first()

    if inventory_item:
        # Si ya la tiene, sumamos 1 al contador de duplicadas
        inventory_item.unpasted_count += 1
    else:
        # Si es nueva, creamos el registro inicial (Ej: Fiat Uno 1.4s Confort 1997)
        new_item = UserInventory(
            user_id=user_id,
            vehicle_id=vehicle.id,
            unpasted_count=1,
            is_pasted=False
        )
        db.add(new_item)

# 4. Confirmar los cambios de forma atómica
db.commit()
```

```
        return {"status": "success", "vehicles_dropped": [v.name for v in new_vehicles]}


    except Exception as e:

        # Si algo falla en el medio (ej. base de datos bloqueada), se deshace todo

        db.rollback()

        raise HTTPException(status_code=500, detail="Error en la transacción. No se descontó la caja.")
```